ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №6

«Автоматизоване тестування Python-програм»

з курсу «Професійний Python»
Варіант №13 — «Система обліку комп'ютерної техніки»

Виконав: Капарник Дмитро Андрійович

Група: ФЕП-32

Перевірив: доцент Куньо Іван Михайлович

Дата: 18.10.2026

Львів — 2026

1. Мета роботи

Створити відтворюваний test suite, який перевіряє звичайні сценарії, межі, помилкові дані та відмови залежностей.

2. Індивідуальний варіант і постановка задачі

Варіант №13 — система обліку комп'ютерної техніки. Протестувати Device/Computer/Printer, реєстрацію й статистику, CSV/JSON, YAML-політики та maintenance service через Mock і AsyncMock. Отримати statement і branch coverage.

Наскрізний проєкт продовжує ЛР №1. Базові поля: інвентарний номер, тип, виробник, модель і вартість. Збережено пошук, фільтр виробника, середню/загальну вартість і визначення найдорожчого пристрою.

3. Структура проєкту та створені модулі

```
inventory-service/
  pyproject.toml, README.md, .gitignore
  src/computer_inventory/
    models.py
    services.py
    analytics.py
    streaming.py
    domain.py
    fileio.py
  src/inventory_service/  # фінальний пакет запуску
  tests/
  scripts/
  docs/GITHUB.md
```

4. Структура test suite та матриця

```
tests/
  conftest.py
  test_inventory.py  # базові unittest-перевірки ЛР1
  test_core.py       # unit та file integration
  test_database.py   # додається в ЛР7
  test_api.py        # додається в ЛР8
  test_performance.py
  test_config.py, test_cli.py
```

Категорія	**Приклади перевірок**	**Очікування**
Normal	Пошук, сума, середнє, maximum, registry	Точний результат
Minimum / maximum	0, 0.01, 10^{12} грн	Допускаються
Invalid	-0.01, NaN, Infinity, 1.001, $>10^{12}$	ValueError / validation error

Категорія	Приклади перевірок	Очікування
Empty / missing	Порожній реєстр, відсутній файл, поле	Нульові підсумки / відповідна помилка
Repeated	Повторна реєстрація номера	Помилка дублювання
Dependency failure	Repository exception, async timeout	Передавання помилки, відсутність небажаного виклику
File integration	CSV → Device → atomic JSON	Точні поля, статистика, збереження попереднього output

5. Fixtures, параметризація та unit tests

```
@pytest.fixture
def devices():
    return create_demo_devices()

@pytest.fixture
def repository():
    return InMemoryRepository[Device]()

@pytest.fixture
def csv_path(tmp_path):
    path = tmp_path / "inventory.csv"
    path.write_text(

"inventory_number,type,manufacturer,model,value\nA,Computer,Dell,Latitude,100.50\nB,Printe
r,HP,LaserJet,200\n",
        encoding="utf-8",
    )
    yield path
    assert path.exists()

@pytest.fixture
def device_factory():
    def factory(number="X", cost="100"):
        return Device(number, "Computer", "Dell", "Latitude", Decimal(cost))

    return factory
```

Fixture csv_path має yield; ресурс окремий для кожного tmp_path. Factory створює Device з потрібними межами. За замовчуванням scope — function. Частина перевірок наведена нижче.

```
@pytest.mark.parametrize(
    "value", ["-0.01", "NaN", "Infinity", "1.001", "1000000000001"]
)
def test_invalid_cost(value, device_factory):
    with pytest.raises(ValueError):
        device_factory(cost=value)

@pytest.mark.parametrize("value", ["0", "0.01", "1000000000000"])
def test_valid_cost_boundary(value, device_factory):
    assert device_factory(cost=value).cost == Decimal(value)
```

```
def test_payload():
    device = from_payload(
        {
            "inventory_number": "A",
            "device_type": "Computer",
            "manufacturer": "Dell",
            "model": "X",
            "cost": "10.10",
        }
    )
    assert float(device.cost) == pytest.approx(10.10)

def test_repository_and_registry(repository, device_factory):
    registry = InventoryRegistry(repository)
    device = device_factory()
    registry.register(device)
    assert len(registry) == 1 and "X" in registry
    assert list(registry) == [device] and registry["X"] == device
    assert registry.best() == device
    assert registry.total.amount == registry.average.amount == Decimal(100)
    assert registry.by_manufacturer("dell") == [device]
    assert len(registry.manufacturers) == 1
    with pytest.raises(ValueError):
        registry.register(device)
    with pytest.raises(KeyError):
        registry["missing"]
```

6. Mock, AsyncMock, monkeypatch та integration tests

```
def test_maintenance_mock(device_factory):
    repository = Mock()
    policy = Mock(spec=StandardMaintenance)
    repository.get.return_value = device_factory()
    policy.interval_days.return_value = 180
    service = MaintenanceService(repository, policy)
    assert service.schedule("X") == 180
    repository.get.assert_called_once_with("X")
    policy.interval_days.assert_called_once_with(repository.get.return_value)
    repository.get.side_effect = RuntimeError("database unavailable")
    policy.reset_mock()
    with pytest.raises(RuntimeError):
        service.schedule("X")
    policy.interval_days.assert_not_called()

def test_missing_maintenance():
    repository = Mock()
    repository.get.return_value = None
    policy = Mock()
    with pytest.raises(LookupError):
        MaintenanceService(repository, policy).schedule("X")
    policy.interval_days.assert_not_called()

def test_async_provider(devices):
    provider = Mock()
```

```
    provider.fetch = AsyncMock(return_value=devices)
    assert asyncio.run(load_remote(provider)) == devices
    provider.fetch.assert_awaited_once()
    provider.fetch.side_effect = TimeoutError("timeout")
    with pytest.raises(TimeoutError):
        asyncio.run(load_remote(provider))

def test_import_export_integration(csv_path, tmp_path):
    import json

    config = FileConfig(csv_path, tmp_path / "out.json")
    stats = ImportStats()
    export_json(import_csv(config, stats), config.output, stats)
    data = json.loads(config.output.read_text(encoding="utf-8"))
    assert len(data) == stats.valid == stats.exported == 2
    assert data[0]["value"] == "100.50"

def test_atomic_failure_integration(csv_path, tmp_path):
    with csv_path.open("a", encoding="utf-8") as stream:
        stream.write("C,Computer,Dell,X,wrong\n")
    config = FileConfig(csv_path, tmp_path / "out.json")
    config.output.write_text("original", encoding="utf-8")
    with pytest.raises(RecordValidationError) as error:
        export_json(import_csv(config, ImportStats()), config.output, ImportStats())
    assert error.value.__cause__ is not None
    assert config.output.read_text(encoding="utf-8") == "original"
    assert not list(tmp_path.glob("*.tmp"))

def test_export_os_failure(tmp_path, monkeypatch):
    from computer_inventory import fileio

    def fail(*args, **kwargs):
        raise OSError("simulated disk failure")

    monkeypatch.setattr(fileio.os, "replace", fail)
    with pytest.raises(ImportExportError):
        export_json(iter([]), tmp_path / "out.json", ImportStats())
    assert not list(tmp_path.glob("*.tmp"))
```

return_value задає успішну відповідь, side_effect моделює відмову. assert_called_once_with перевіряє аргументи, assert_not_called — заборонену дію після помилки. AsyncMock перевіряється через assert_awaited_once. monkeypatch тимчасово замінює os.replace для імітації помилки диска. Tests не залежать від інтернету.

7. Результати та coverage

```
.................................................           [100%]
============================== tests coverage ==============================
_____________ coverage: platform win32, python 3.12.14-final-0 _____________

Name                                  Stmts   Miss Branch BrPart  Cover   Missing
---------------------------------------------------------------------------------
src\computer_inventory\__init__.py        1      0      0      0   100%
```

```
src\computer_inventory\analytics.py      52      0      8      0   100%
src\computer_inventory\domain.py        106      1     16      1    98%   90, 114->exit
src\computer_inventory\fileio.py        139      6     34      1    96%   57-62, 99
src\computer_inventory\main.py           93     29     28      6    69%   30-31, 51, 77-
86, 104-105, 107, 111-112, 116, 122-133, 137
src\computer_inventory\models.py         19      0      8      0   100%
src\computer_inventory\services.py       25      0      6      0   100%
src\computer_inventory\streaming.py      75      0     24      1    99%   97->92
------------------------------------------------------------------------------
TOTAL                                   510     36    124      9    93%
Coverage HTML written to dir ../../work/coverage06
Coverage JSON written to file ../../work/coverage06.json
52 passed, 9 subtests passed in 1.46s
```

На етапі ЛР6 отримано 52 passed і 9 успішних subtests. Branch coverage увімкнено; сукупне покриття становило 93%. Після розширення проєкту всіма лабораторними фінальна перевірка отримала 84 passed, 9 subtests і 96,45% сукупного покриття. Частина консольних інтерактивних гілок і тонкий модуль запуску перевірені демонстраціями, але не всі ці переходи враховані в pytest coverage.

Метрика фінального suite	**Значення**
Покрито інструкцій	910 / 931
Покрито гілок	177 / 196
Сукупний відсоток	96.45%
Поріг CI	80%

HTML-звіт coverage містить таблицю файлів, відсотки та посилання на підсвічені пропуски. Його збережено разом з експериментальними результатами; для нового запуску: pytest --cov=computer_inventory --cov=inventory_service --cov-branch --cov-report=html. Найнижче модульне покриття має консольна точка входу; для async-клієнта залишилися окремі помилкові формати відповіді та вичерпання повторів.

8. Експерименти якості тестування

Експеримент	**Фактичний результат**	**Пояснення**
Слабка / сильна перевірка	Виклик wrong_total пройшов; assert виявив неправильну суму	Виконання не дорівнює перевірці
Coverage без assertions	Навмисно неправильний результат лишився непоміченим до assert	Покриття не вимірює правильність
Branch coverage	1/2 → 2/2 гілок	Додавання випадку для іншої гілки
Mock / fake	Однаковий maintenance interval	Mock зручний для interactions; fake — для стану
Нестабільна перевірка	15 passed, 5 failed із 20	Залежність від послідовності random

Експеримент	Фактичний результат	Пояснення
Фіксований seed	20 / 20 passed	Керований результат
Файлова ізоляція	Два різні temporary directories, обидва очищені	Відсутність спільного output
Failure path	Repository failure і async timeout перевірені	Policy не викликається після відмови repository

```
def wrong_total(values):
    return sum(values) + 1
```

Навмисно дефектні приклади ізольовано від основного suite. Під час фінальної перевірки узгодженості виявлено реальний крайовий випадок: середнє для цін 0.00 і 0.01 округлювалося по-різному в сервісах, streaming та БД. Правило уніфіковано до ROUND_HALF_UP; test_rounding_consistency фіксує результат 0.01 в обох нових шарах.

Код і історія лабораторних: https://github.com/dimakaparnyk/professional-python-labs. Для відповідного етапу використовується тег lab-06, фінальна інтегрована реалізація — main (перевірений CI commit 2e3fb3d).

9. Відповіді на контрольні запитання

1. Що таке automated test?

Програма, яка автоматично виконує сценарій і перевіряє очікувані властивості результату.

2. Які основні переваги автоматизованого тестування?

Повторюваність, швидкий зворотний зв’язок, захист від регресій і документування поведінки.

3. Що таке regression?

Порушення раніше коректної поведінки після внесення змін.

4. Що таке unit test?

Перевірка невеликого компонента в ізоляції від дорогих або нестабільних залежностей.

5. Що таке integration test?

Перевірка взаємодії кількох реальних компонентів, наприклад імпорту та файлового експорту.

6. Чим integration test відрізняється від unit test?

Integration test охоплює зв’язки між компонентами, unit test зосереджується на вузькій поведінці.

7. Що показує test pyramid?

Багато швидких unit tests, менше integration tests і небагато найдорожчих наскрізних перевірок.

8. Чому unit tests зазвичай більше?

Вони швидші, точніше локалізують дефект і дешевші у супроводі.

9. Що таке pytest?

Фреймворк тестування Python з fixtures, параметризацією та зручними повідомленнями про помилки.

10. Як pytest знаходить tests?

За домовленими назвами test_*.py, функціями test_* і методами тестових класів.

11. Якою командою запускається pytest?

python -m pytest з каталогу проєкту.

12. Для чого pytest -v?

Для докладного переліку тестів та їхніх результатів.

13. Для чого pytest -x?

Щоб зупинити запуск після першого невдалого тесту.

14. Що означає Arrange–Act–Assert?

Підготовка передумов, виконання дії та перевірка результату.

15. Для чого використовується assert?

Для перевірки логічної умови; хибна умова спричиняє AssertionError.

16. Що таке assertion failure?

Невідповідність фактичної поведінки очікуванню, сформульованому в assert.

17. Чому floats не завжди слід порівнювати через ==?

Деякі десяткові значення не мають точного двійкового представлення, тому можливі похибки округлення.

18. Для чого pytest.approx?

Для порівняння чисел із допустимою відносною або абсолютною похибкою.

19. Для чого pytest.raises?

Для перевірки, що конкретна операція піднімає очікуваний виняток.

20. Як перевірити exception message?

Передати match у pytest.raises або перевірити str(exc_info.value).

21. Як отримати exception object через exc_info?

Зберегти контекст через with pytest.raises(...) as exc_info та використати exc_info.value.

22. Що таке boundary testing?

Перевірка значень на межах дозволеного діапазону та безпосередньо поруч із ними.

23. Які boundaries потрібно тестувати для діапазону 0–100?

0 і 100 як допустимі межі, значення всередині та нижче 0 і вище 100.

24. Що таке parametrization?

Запуск однієї тестової функції для кількох наборів вхідних даних.

25. Для чого pytest.mark.parametrize?

Для оголошення наборів параметрів і очікуваних результатів тесту.

26. Що дають ids параметризованих tests?

Зрозумілі назви окремих випадків у результатах та повідомленнях про помилки.

27. Що таке fixture?

Іменований постачальник тестових даних або ресурсу з керованим життєвим циклом.

28. Для чого використовуються fixtures?

Для повторного використання підготовки даних, ізоляції та звільнення ресурсів.

29. Що означає fixture setup?

Створення передумов і ресурсу перед тестом.

30. Що означає fixture teardown?

Очищення ресурсу після тесту, зокрема якщо перевірка завершилася невдало.

31. Для чого yield у fixture?

Видає ресурс тесту; код після yield виконує очищення.

32. Які fixture scopes підтримує pytest?

function, class, module, package, session.

33. Який scope є default?

function — нове значення fixture для кожної тестової функції або випадку.

34. Для чого conftest.py?

Для спільних fixtures і налаштувань тестів у певному каталозі та його підкаталогах.

35. Чому fixtures із conftest.py не імпортують вручну?

pytest автоматично знаходить їх у відповідній області й передає за іменами параметрів.

36. Що таке tmp_path?

Fixture з унікальним тимчасовим каталогом як об’єктом Path.

37. Чому tmp_path корисний?

Дозволяє перевіряти реальні файли без конфліктів і пошкодження даних проєкту.

38. Що таке test isolation?

Незалежність результату тесту від порядку, інших тестів та зовнішнього стану.

39. Для чого monkeypatch?

Для тимчасової заміни атрибутів, середовища або робочого каталогу з автоматичним відновленням.

40. Як monkeypatch змінює environment variable?

Через monkeypatch.setenv("NAME", "value").

41. Як видалити environment variable у test?

Через monkeypatch.delenv("NAME", raising=False).

42. Як підмінити function через monkeypatch?

Через monkeypatch.setattr(module, "function", replacement) у місці використання функції.

43. Що таке test double?

Замінник реальної залежності для керування умовами тесту.

44. Що таке Dummy?

Об’єкт-заповнювач, який передається для контракту, але не використовується сценарієм.

45. Що таке Stub?

Замінник, що повертає заздалегідь задані відповіді.

46. Що таке Fake?

Спрощена працездатна реалізація, наприклад репозиторій у пам’яті.

47. Що таке Mock?

Замінник, що записує взаємодії та дозволяє перевірити виклики.

48. Що таке Spy?

Обгортка або замінник, який спостерігає за викликами, іноді зберігаючи реальну поведінку.

49. Для чого Mock?

Для керування результатом залежності й перевірки потрібних або небажаних взаємодій.

50. Що таке spec?

Контракт доступних атрибутів Mock; допомагає виявити звернення до неіснуючого методу.

51. Для чого return_value?

Задає значення, яке повертає виклик Mock.

52. Для чого side_effect?

Може задавати виняток, послідовність результатів або функцію поведінки Mock.

53. Як перевірити, що method викликано один раз?

Через mock.method.assert_called_once() або assert_called_once_with(...).

54. Як перевірити parameters виклику?

Через assert_called_with(...), assert_called_once_with(...) або аналіз call_args.

55. Як перевірити, що method не викликався?

Через mock.method.assert_not_called().

56. Що таке state testing?

Перевірка підсумкового стану та повернених значень.

57. Що таке interaction testing?

Перевірка того, які залежності були викликані, з якими аргументами та скільки разів.

58. Коли interaction testing виправдане?

Коли сама взаємодія є контрактом: повідомлення, оплата, запис або заборона виклику після помилки.

59. Що таке patch?

Тимчасова заміна атрибута чи імені під час виконання тесту.

60. Чому patch потрібно виконувати там, де name використовується?

Код звертається до свого імпортованого імені; заміна вихідного визначення може не змінити вже імпортоване посилання.

61. Що таке AsyncMock?

Замінник асинхронного callable, результат якого можна await та перевіряти окремо від виклику.

62. Як перевірити awaited call?

Через assert_awaited_once_with(...) або assert_awaited_once().

63. Що таке flaky test?

Тест, що за незмінного коду нестабільно проходить і падає.

64. Чому flaky tests небезпечні?

Підриває довіру до перевірок і маскує реальні регресії.

65. Що таке deterministic test?

Тест із керованими входами, часом і залежностями та відтворюваним результатом.

66. Чому unit test не повинен залежати від Internet?

Мережа додає затримки, недоступність і сторонні зміни; для unit test її замінюють керованою залежністю.

67. Коли в integration test доцільний real filesystem?

Коли перевіряється реальна серіалізація, шлях, atomic replace або взаємодія файлових компонентів.

68. Чому не потрібно mock-ати все?

Суцільні заміни можуть приховати помилки взаємодії та тестувати лише налаштовані mocks.

69. Що таке over-mocking?

Надмірна підміна реалізації, через яку тести стають крихкими та відірваними від поведінки.

70. Що таке coverage?

Метрика частки коду або шляхів, виконаних під час тестів.

71. Що таке statement coverage?

Частка виконаних інструкцій або рядків серед тих, що враховуються інструментом.

72. Що таке branch coverage?

Частка виконаних переходів між гілками керування.

73. Чим вони відрізняються?

Виконаний if може дати покритий рядок, хоча одна з його гілок жодного разу не перевірялася.

74. Як отримати coverage через pytest-cov?

python -m pytest --cov=computer_inventory --cov-branch.

75. Для чого --cov-report=term-missing?

Для переліку рядків і переходів, які не виконувалися.

76. Для чого HTML coverage report?

Для навігації по покриттю кожного файла з підсвічуванням пропущених ділянок.

77. Чи гарантує 100% coverage відсутність bugs?

Ні; виконання рядка не гарантує правильність результату чи повноту вимог.

78. Чому meaningful assertions важливіші за coverage percentage?

Вони перевіряють контракт результату, тоді як coverage лише показує виконання коду.

79. Що таке coverage threshold?

Мінімальний допустимий відсоток, нижче якого запуск перевірок вважається невдалим.

80. Чому tests повинні бути незалежними один від одного?

Щоб порядок і спільні ресурси не змінювали результат та тести можна було запускати окремо.

10. Висновок

Побудовано незалежний suite із fixtures, параметризацією, перевірками винятків, реальними файловими integration tests та керованими залежностями. Експерименти підтвердили важливість змістовних assert, перевірки гілок і детермінізму.